Autonomous Leveraged ETF Trading Bot

Full Project Architecture, Mathematical Strategies & New Laptop Setup Guide

System Overview: *This autonomous quantitative trading system executes algorithmic momentum strategies across a 36-ETF universe of high-beta 2x and 3x leveraged bull and bear exchange-traded funds. It combines an institutional* ***Macro Cash Shield****, dynamic* ***ATR Trailing Stops****,* ***Profit-Tier Ratchets****, real-time* ***VADER News Sentiment Scoring*** *with automated* ***Long/Short Pair Switching****, and a live* ***Web Operations Cockpit (http://localhost:5000)*** *for monitoring and portfolio management.*

1. Project Architecture & File-by-File Breakdown

The system is designed with strict modular separation between order execution, news intelligence, user interface, and backtesting harnesses:

File Name	Primary Role & Core Functions
live_trader.py	**Live Execution Engine**: Connects to Alpaca Paper/Live API using LumiBot. Evaluates candidate ETFs, executes daily champion rotation, tracks peak prices for trailing stops, enforces Macro Cash Shield, and triggers breaking news inverse switches.
news_sentiment_engine.py	**AI News Intelligence**: Connects to Alpaca News API (Benzinga, Bloomberg, Dow Jones). Performs real-time VADER lexicon sentiment polarity scoring (-1.0 to +1.0). Maps 9 sector Long/Short pairs (e.g., SOXL <-> SOXS, TQQQ <-> SQQQ) and caches signals.
dashboard.py	**Web Operations Cockpit (Port 5000)**: Built with native Python HTTP daemon and Vanilla CSS/JS. Displays account equity curve (Chart.js), open positions PnL, executed trades, sector sentiment pills, breaking headlines feed, single-click ticker modal, and emergency liquidate button.
run_champion_backtest.py	**Historical Simulation Harness**: Matches live execution rules 1-to-1. Replays historical daily bars, tracks portfolio equity indicators, and outputs interactive QuantStats HTML tear sheets, indicators charts, and trade execution logs in logs/.
watchlist.json	**Active Universe Registry**: Central synchronized JSON store holding the active candidate ETFs/stocks. Supports live dynamic additions and deletions from the Web UI modal without rebooting the bot.
.env	**Secure Configuration**: Holds Alpaca API keys, secret tokens, paper trading toggles, and base URLs. Never committed to version control.

2. Quantitative Strategies & Mathematical Risk Controls

To eliminate the steep givebacks common to 2x/3x leveraged ETFs, the bot employs four layers of protective engineering:

A. Macro Cash Shield (Regime Filter):

Checks the benchmark (SPY) against its 50-day Simple Moving Average. If SPY < 50 SMA, the bot instantly liquidates all equity positions and sits in **100% Cash**. In backtests, this kept capital safe for 42% of the year, preventing catastrophic market pullbacks.

B. 20-Day SMA Trend & 10-Day Momentum (ROC) Ranking:

A candidate ETF must be trading above its 20-day SMA to qualify. Qualified candidates are ranked by 10-day Rate of Change: ROC = (Current - Price_10d_ago) / Price_10d_ago. Only the **Top 3 Champions** are bought, with ~28% allocated to each (84% total exposure, 16% safety cash).

C. Dynamic ATR Trailing Stops & Profit-Tier Ratchet:

Volatility stops adapt using the 14-day Average True Range: Stop_Dist = Clamp(3.5%, 6.0%, (1.8 * ATR_14) / Price).

When a position surges parabolically, the **Profit Ratchet** tightens stops: at **+15% profit**, stop tightens to **3.0%**; at **+35% profit**, stop locks at **2.0%** from peak to exit right at market tops.

D. Real-Time News Sentiment & Inverse Pair Switching:

The bot scans breaking wire headlines. If a sector's sentiment score turns severely bearish (≤ -0.18), the bot immediately dumps the Bull ETF and buys the corresponding **Inverse Bear ETF** (e.g., exits SOXL and buys SOXS) to profit while the sector crashes.

Active Long / Short Sector Pairs:

Sector Domain	Bull ETF (Long)	Bear ETF (Inverse Short)	Underlying Proxy Tickers
Semiconductors & AI	SOXL (3x)	SOXS (3x)	NVDA, AMD, TSM, INTC
Tech & Nasdaq 100	TQQQ (3x)	SQQQ (3x)	AAPL, MSFT, GOOGL, QQQ
S&P; 500 Broad Market	UPRO (3x)	SPXU (3x)	SPY, VOO, Large-Cap Index
Financials & Banks	FAS (3x)	FAZ (3x)	JPM, BAC, XLF
Energy & Oil Exploration	GUSH (2x)	DRIP (2x)	XOP, CVX, XOM
Biotech & Genomics	LABU (3x)	LABD (3x)	XBI, IBB
Nvidia Ecosystem	NVDL (2x)	NVDD (2x)	NVDA single-stock leverage
Tesla & EV	TSLL (2x)	TSLS (2x)	TSLA single-stock leverage

3. New Laptop Installation & Setup Guide

Follow this step-by-step checklist to transfer and run the complete trading system on any brand-new Windows, macOS, or Linux laptop.

Step 1: Install Python (Version 3.11 or 3.12 64-Bit)

1. Download Python from https://www.python.org/downloads/ (version 3.11 or 3.12 recommended).
2. **CRITICAL (Windows):** During setup, check the box: **'Add python.exe to PATH'**.
3. Open Command Prompt or PowerShell and verify: python --version and pip --version.

Step 2: Transfer / Clone the Project Folder

Copy the project folder my_algo_bot to your preferred drive on the new laptop (e.g., C:\alpaca\my_algo_bot or D:\alpaca\my_algo_bot).

Step 3: Create an Isolated Python Virtual Environment

Open terminal in the project directory and run:

```
cd C:\alpaca\my_algo_bot
python -m venv venv
```

Step 4: Activate the Virtual Environment

Windows (PowerShell):

```
.\venv\Scripts\Activate.ps1
# If PowerShell blocks script execution, run this once:
# Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

macOS / Linux:

```
source venv/bin/activate
```

Step 5: Install Required Dependencies

Install the exact trading framework, financial analytics, and AI sentiment packages:

```
pip install --upgrade pip
pip install -r requirements.txt
pip install nltk reportlab
python -c "import nltk; nltk.download('vader_lexicon')"
```

Step 6: Configure Environment Credentials (.env)

Create or edit the .env file in the project root directory with your Alpaca paper API credentials:

```
ALPACA_API_KEY=""
ALPACA_API_SECRET=""
ALPACA_IS_PAPER="true"
APCA_API_BASE_URL="https://paper-api.alpaca.markets/v2"
```

Step 7: Launch the Web Cockpit & Live Trader

You can launch the Web Cockpit standalone or start the autonomous bot:

```
# Option A: Start Web Dashboard only:
python dashboard.py
# -> Open your browser to http://localhost:5000

# Option B: Launch Live Autonomous Bot (automatically boots Dashboard on port 5000):
```

```
python live_trader.py
```

Step 8: Run Historical Backtests

To simulate and evaluate performance over custom date ranges (e.g., previous 1-year or multi-year windows):

```
python run_champion_backtest.py
# -> Generates interactive HTML tear sheets in the logs/ folder.
```

4. Troubleshooting & Operational FAQ

Q: Port 5000 Already in Use?

If another background process occupies port 5000, find and terminate it via PowerShell:
Get-NetTCPConnection -LocalPort 5000 | Select-Object -ExpandProperty OwningProcess | Stop-Process -Force

Q: Market CLOSED / Bot Sleeping?

US Equity and Leveraged ETF exchanges trade 9:30 AM – 4:00 PM EST, Monday through Friday. Outside these hours, the bot sleeps to prevent high slippage and illiquid execution spreads.

Q: Does the Bot Buy All Watchlist Stocks?

No. The watchlist serves as candidate radar. The bot filters candidates by trend (20 SMA) and sentiment, then purchases **only the Top 3 Champions** (~28% each), keeping the remaining funds in cash reserve.

Q: How to Add New Tickers on the Fly?

Open http://localhost:5000 in your browser and click **'+ Add Stock Ticker'**. Type any ticker (e.g. NVDA, TQQQ, AAPL) and submit. The system updates watchlist.json instantly with duplicate protection.